

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

feature/data-ui

Screenshot:

- Terminal showing your current branch

```
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol\my_first_flutter_app> git branch
* feature/data-ui
  feature/widgets
  main
```



Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Screenshot:

- Your identified data
- The corresponding data types

String	name
Int	age
Double	height
bool	status

🚩 Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

Create variables for at least:

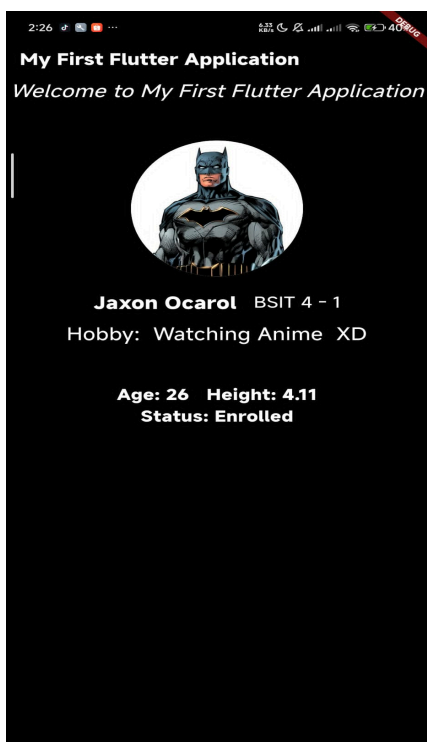
- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

Screenshot:

- Your variables
- Their data types
- Application still running

```
3
4 class _MyHomePageState extends State<MyHomePage> {
5
6   String name = 'Jaxon Ocarol';
7   String course = 'BSIT 4 - 1';
8   String hobby = ' Watching Anime XD';
9   int age = 26;
0   double height = 4.11;
1   bool studentstatus = true;
2
```



Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

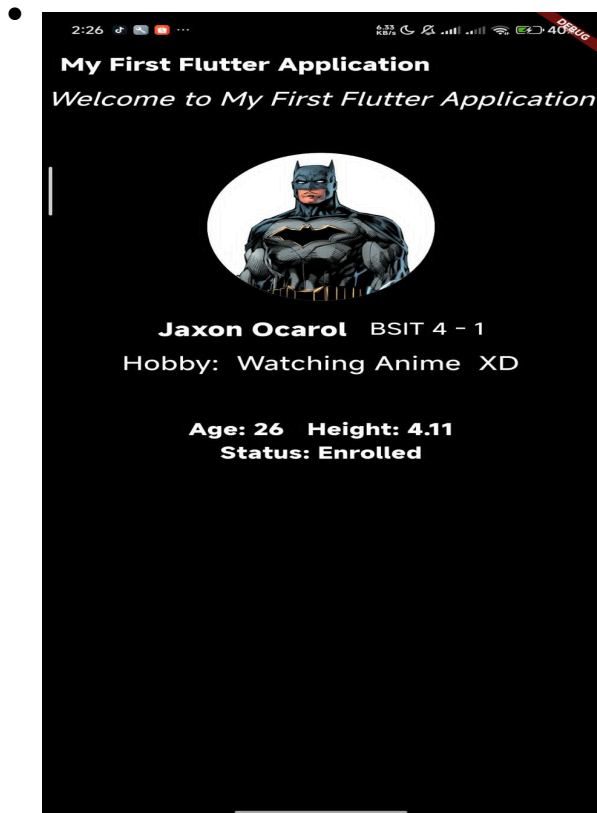
- Variables

```
3
4 class _MyHomePageState extends State<MyHomePage> {
5
6   String name = 'Jaxon Ocarol';
7   String course = 'BSIT 4 - 1';
8   String hobby = ' Watching Anime XD';
9   int age = 26;
0   double height = 4.11;
1   bool studentstatus = true;
2
```

- Widgets displaying the variables

```
1 // Name and Course
2 Row(
3   mainAxisAlignment: MainAxisAlignment.center,
4   children: [
5     Text(
6       name,
7       style: const TextStyle(
8         fontSize: 24,
9         fontWeight: FontWeight.bold,
10        color: Colors.white,
11      ), // TextStyle
12    ), // Text
13    const SizedBox(width: 20),
14    Text(
15      course,
16      style: const TextStyle(fontSize: 20, color: Colors.white),
17    ), // Text
18  ],
19 ), // Row
20
21 const SizedBox(height: 10),
22
23 // Hobby
24 Text(
25   'Hobby: $hobby',
26   style: const TextStyle(fontSize: 24, color: Colors.white),
27   textAlign: TextAlign.center,
28 ), // Text
29
30 const SizedBox(height: 20)
```

- Updated application



🚩 Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

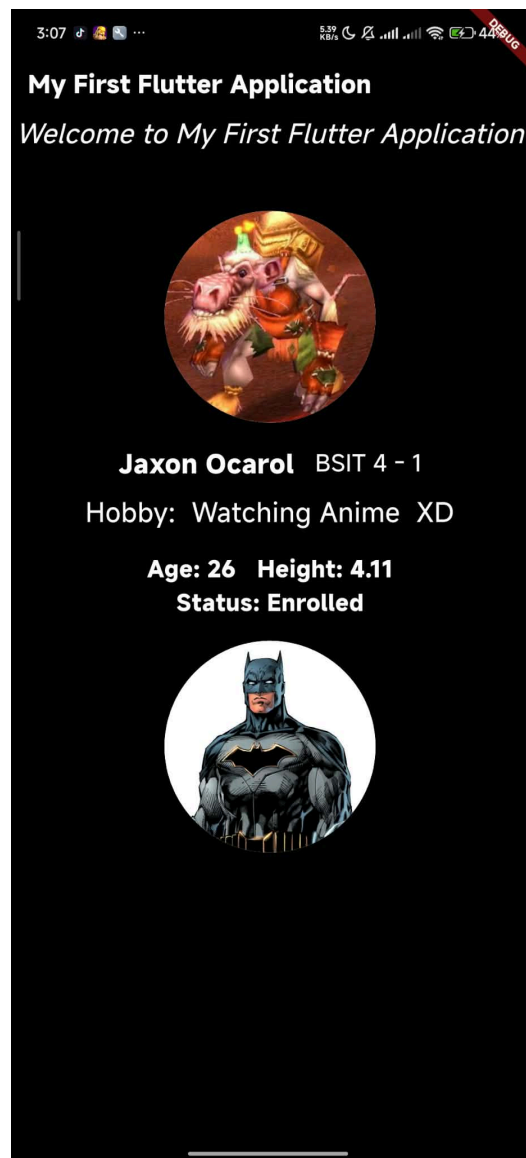
- Image variable

```
final String imagePath = 'assets/images/meepo.jpg';  
final String imagePath2 = 'assets/images/batman.jpg';
```

- Image widget using the variable

```
CircleAvatar(
  radius: 100,
  backgroundColor: Colors.white,
  backgroundImage: const AssetImage('assets/images/meepo.jpg'),
),
const SizedBox(height: 20),
CircleAvatar(
  radius: 100,
  backgroundColor: Colors.white,
  backgroundImage: const AssetImage('assets/images/batman.jpg'),
),
```

- Updated application



🚩 Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

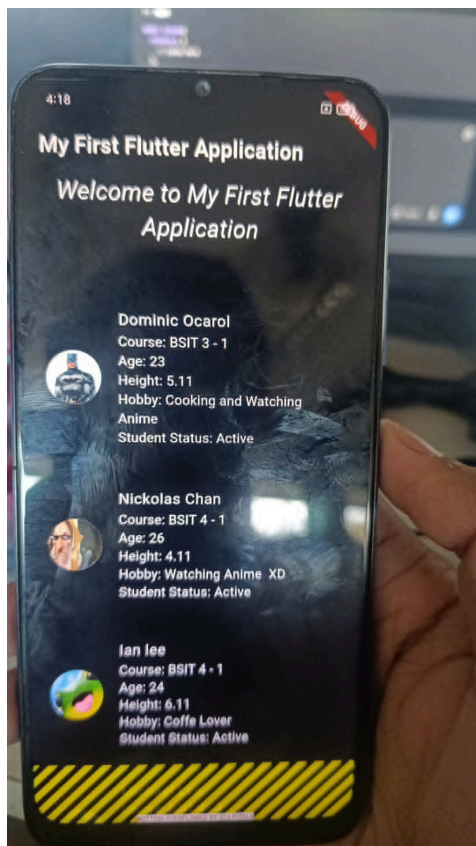
Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data



```
//Profile Card 4
Card(
  color: Color.fromARGB(255, 255, 255, 255),
  child: ListTile(
    leading: CircleAvatar(
      radius: 30,
      backgroundColor: Colors.white,
      backgroundImage: AssetImage('assets/images/fat.jpg'),
    ), // CircleAvatar
    title: Text(
      profile3.name,
      style: const TextStyle(color: Colors.black),
    ), // Text
    subtitle: Text(
      'Course: $profile3.course\nAge: $profile3.age\nHeight: $profile3.height\nHobby: $profile3.hobby\nStudent Status: $profile3.studentstatus ? 'Active' : 'Inactive',
      style: const TextStyle(color: Colors.black),
    ), // Text
  ), // ListTile
), // Card
const SizedBox(height: 20),

//Profile Card 5
Card(
  color: Color.fromARGB(255, 255, 255, 255),
  child: ListTile(
    leading: CircleAvatar(
      radius: 30,
      backgroundColor: Colors.white,
      backgroundImage: AssetImage('assets/images/mepo.jpg'),
    ), // CircleAvatar
    title: Text(
      profile4.name,
      style: const TextStyle(color: Colors.black),
    ), // Text
    subtitle: Text(
      'Course: $profile4.course\nAge: $profile4.age\nHeight: $profile4.height\nHobby: $profile4.hobby\nStudent Status: $profile4.studentstatus ? 'Active' : 'Inactive',
      style: const TextStyle(color: Colors.black),
    ), // Text
  ), // ListTile
), // Card
), // Column
), // Scaffold
```

```
//Profile Card 4
Card(
  color: Color.fromARGB(255, 255, 255, 255),
  child: ListTile(
    leading: CircleAvatar(
      radius: 30,
      backgroundColor: Colors.white,
      backgroundImage: AssetImage('assets/images/fat.jpg'),
    ), // CircleAvatar
    title: Text(
      profile3.name,
      style: const TextStyle(color: Colors.black),
    ), // Text
    subtitle: Text(
      'Course: $profile3.course\nAge: $profile3.age\nHeight: $profile3.height\nHobby: $profile3.hobby\nStudent Status: $profile3.studentstatus ? 'Active' : 'Inactive',
      style: const TextStyle(color: Colors.black),
    ), // Text
  ), // ListTile
), // Card
const SizedBox(height: 20),

//Profile Card 5
Card(
  color: Color.fromARGB(255, 255, 255, 255),
  child: ListTile(
    leading: CircleAvatar(
      radius: 30,
      backgroundColor: Colors.white,
      backgroundImage: AssetImage('assets/images/mepo.jpg'),
    ), // CircleAvatar
    title: Text(
      profile4.name,
      style: const TextStyle(color: Colors.black),
    ), // Text
    subtitle: Text(
      'Course: $profile4.course\nAge: $profile4.age\nHeight: $profile4.height\nHobby: $profile4.hobby\nStudent Status: $profile4.studentstatus ? 'Active' : 'Inactive',
      style: const TextStyle(color: Colors.black),
    ), // Text
  ), // ListTile
), // Card
), // Column
), // Scaffold
```



Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

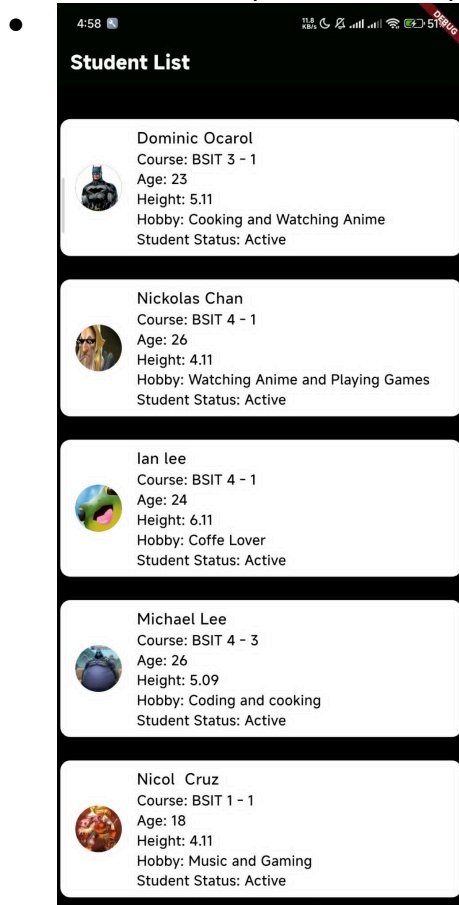
without changing the size of the screen.

Screenshot:

- The scrolling widget in your code

```
.18   body: SingleChildScrollView(  
.19     child: Column(  
.20       mainAxisAlignment: MainAxisAlignment.start,  
.21       children: [  
.22         //Application Title
```

- Top portion of the application
- The rest of the portion of the application showing the later profiles



Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: **missing**

Profile B

Name: Juan

Course: **missing**

Hobby: Gaming

Profile C

Name: **missing**

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

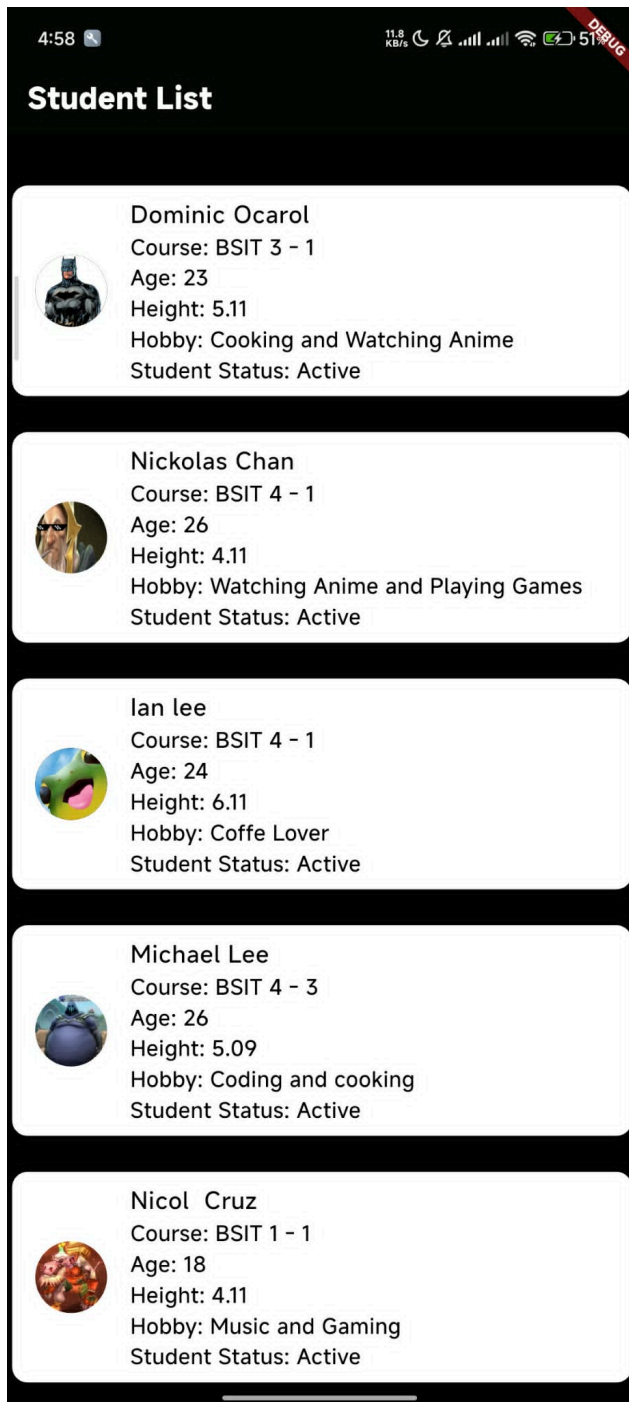
- Incomplete profile data

```
54 Profile profile = Profile(  
55     name: 'Dominic Ocarol',  
56     course: 'BSIT 3 - 1',  
57     age: 23,  
58     height: 5.11,  
59     hobby: 'Cooking and Watching Anime',  
60     image: 'assets/images/batman.jpg',  
61     studentstatus: true,  
62 );  
63  
64 Profile profile1 = Profile(  
65     name: 'John Doe',  
66     age: 26,  
67     height: 4.11,  
68     hobby: 'Watching Anime and Playing Games',  
69     image: 'assets/images/meepo.jpg',  
70     studentstatus: true,  
71 );  
72  
73 Profile profile2 = Profile(  
74     name: 'Jane Smith',  
75     course: 'BSIT 4 - 1',  
76     age: 24,  
77     height: 6.11,  
78     hobby: 'Coffe Lover',  
79     image: 'assets/images/meepo.jpg',  
80     studentstatus: true,  
81 );  
82 Profile profile3 = Profile(  
83     name: 'Michael Lee',  
84     course: 'BSIT 4 - 3',  
85     age: 26,  
86     height: 5.09,  
87     hobby: 'Coding and cooking',  
88     image: 'assets/images/meepo.jpg',  
89     studentstatus: true,  
90 );
```

- Application displaying fallback values



- Application still functioning correctly





Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

feature/data-ui

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol\my_first_flutter_app> git branch
* feature/data-ui
  feature/widgets
  main
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol\my_first_flutter_app> git add my_first_flutter_app
fatal: pathspec 'my_first_flutter_app' did not match any files
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol\my_first_flutter_app> cd C:\Users\dominic\source\repos\IT\IT314AB_Ocarol
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol> git add my_first_flutter_app
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol> git commit -m "FEAT: Added data ui and fallback values"
[feature/data-ui 31c44cf] FEAT: Added data ui and fallback values
 6 files changed, 168 insertions(+), 96 deletions(-)
 create mode 100644 my_first_flutter_app/assets/images/bard.jpg
 create mode 100644 my_first_flutter_app/assets/images/fat.jpg
 create mode 100644 my_first_flutter_app/assets/images/meepo.jpg
 create mode 100644 my_first_flutter_app/assets/images/tuglife.jpg
PS C:\Users\dominic\source\repos\IT\IT314AB_Ocarol> git push origin feature/data-ui
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), 102.42 KiB | 7.88 MiB/s, done.
Total 12 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/data-ui' on GitHub by visiting:
remote:   https://github.com/domsprog/IT314AB_Ocarol/pull/new/feature/data-ui
remote:
To https://github.com/domsprog/IT314AB_Ocarol.git
 * [new branch]      feature/data-ui -> feature/data-ui
```